Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 6

Users, Groups and File Permissions

Lab 6 guide: read the mode bits, set the least access that works, and follow the whole path to a file.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Read owner, group and other mode bits, in symbolic and octal notation.
2. Set least-privilege modes on files and directories you own, and explain directory traversal.
3. Diagnose an access failure from path and mode evidence, and separate what you observed from what you only reasoned.

THIS LAB

Lab 6

First the concepts, with examples. Then the lab on the server, using only files you own. Accounts, ACLs and special bits are optional extensions, so they come last.

DISCUSSION

A file is readable by everyone. Can everyone read it?

What if the folder that holds the file is private? Where does the check really start?

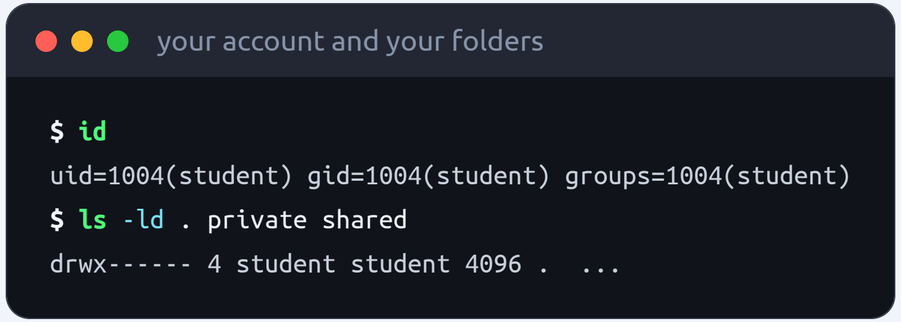
CONCEPTS FIRST

Who may do what to a file

Three ideas: three kinds of user, three kinds of access, and a check on every folder of the path.

Users and groups

- Every file has an owner (a user) and an owning group.
- Every process runs as a user, and has a primary group and sometimes more.
- The kernel compares who is asking with the owner, the group and everybody else, in that order.
- The first match decides. A normal user cannot give a file to another account.



Reading the mode string

student@server: ~/oslab-work/lab6

```
$ ls -l report.txt
-rw-r----- 1 student staff 120 report.txt
```

Characters	Part	Here
`-`	Type: - file, d directory, l link	A plain file
`rw-`	Owner	Read and write
`r--`	Owning group	Read only
`---`	Everybody else	No access

The first character is the type. The next nine are three triples: owner, group, others. A dash means that access is not given.

Octal and symbolic modes

Bit	Value	On a file	On a directory
r	4	Read the contents	List the names inside
w	2	Change the contents	Create, rename or remove entries
x	1	Run it as a program	Search: pass through to what is inside

Add the values of one triple to get one digit. rw- is 4+2 = 6, r-- is 4. So rw-r----- is 640.

chmod, in two notations

student@server: ~/oslab-work/lab6

```
$ chmod 600 note.txt
$ chmod u=rw,g=r,o= note.txt
$ stat -c '%A %a %n' note.txt
-rw-r----- 640 note.txt
$ chmod g-r note.txt
$ chmod u+x script.sh
```

Octal sets all nine bits at once. Symbolic mode changes only what you name: u owner, g group, o others, = sets, + adds, - removes.

stat -c shows the mode in both forms: %A symbolic, %a octal.

On a directory, x means search

READ

r on a directory

You may list the names inside it, with ls.

SEARCH

x on a directory

You may pass through it to reach an entry whose name you already know. Without x you cannot open anything below it.

TOGETHER

Both are needed to browse

To list names and open files you need r and x. A directory with only r shows names but opens nothing.

Do not try to run a data file to test read permission. For a directory, x is not about running a program.

The whole path is checked

student@server: ~/oslab-work/lab6

```
$ namei -l "$PWD/demo/note.txt"
f: /home/student/oslab-work/lab6/demo/note.txt
drwxr-xr-x root    root    /
drwxr-xr-x root    root    home
drwx------ student student student
drwx------ student student oslab-work
-rw-r----- student student note.txt
```

To open a file you need x on every directory of the path, from / down to the file.

A file with mode 644 inside a 700 directory is out of reach for other users. The strictest folder wins.

If namei is missing, list each parent with ls -ld.

LAB 6

A private record and a public notice

Configure your own copy first, and explain the access path before anyone prepares a multi-user exercise.

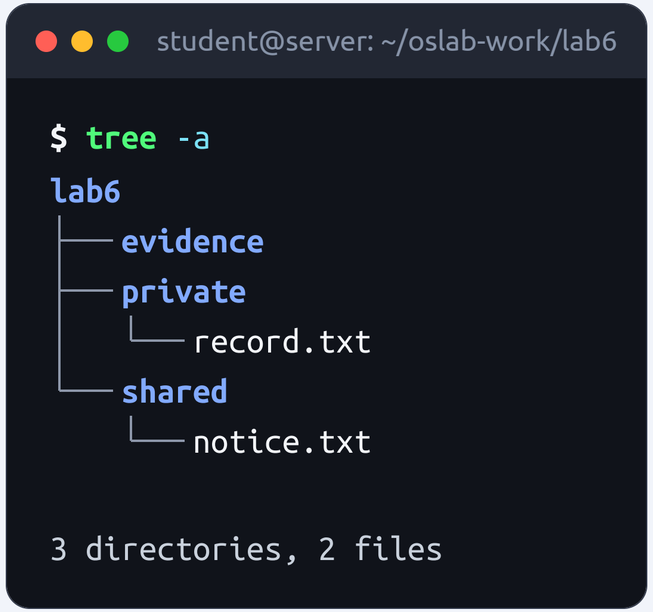
The scenario

SCENARIO

Records at TechCorp

TechCorp needs a private record and a notice that staff can eventually read. Configure your owned copy first and justify the whole access path.

You work only on files you own. Any real multi-user test needs an account the instructor has prepared.



Inside the 120 minutes

Minutes	What happens
0 to 10	Introduction, objectives and setup
10 to 25	Guided example: permission basics
25 to 35	Written prediction
35 to 70	Task 3: policy, modes and traversal

Minutes	What happens
70 to 85	Owner, search and peer-access tests
85 to 100	Individual changed-case checkpoint
100 to 110	Correction and explanation
110 to 120	Cleanup and submission

How you work

Your own account

Log in with your own account. Use only your own files and processes. Never edit sudoers or create accounts for the core. Do not widen your home folder to test a peer.

AI tools

Optional in the investigation and the tests. Not allowed in the prediction and the individual checkpoint.

Evidence

Keep what you observe in the evidence folder. Short, attributable evidence earns more than long output.

Initial prediction accuracy is lightly weighted. A reasoned attempt and an evidence-based correction earn credit.

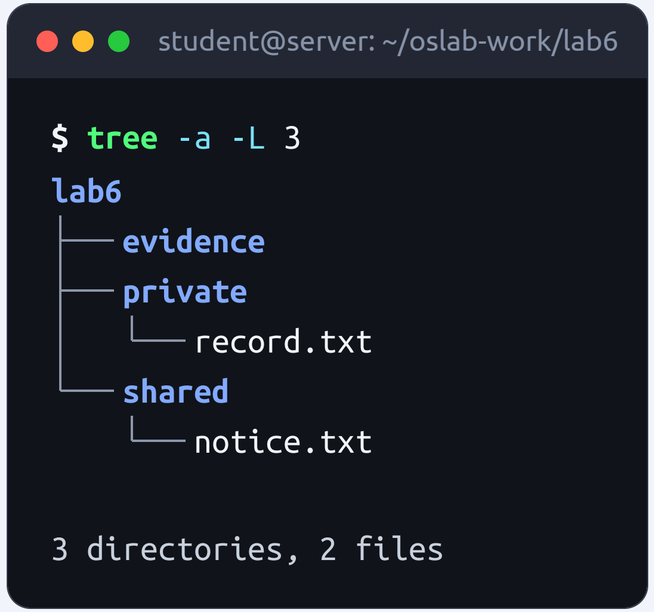
Setup: start the lab

student@server: ~

```
$ whoami
$ command -v oslab
$ oslab doctor
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab6
$ cd "$OSLAB_WORKSPACE/lab6"
$ pwd
$ mkdir -p evidence
$ find . -maxdepth 3 -type f
```

- whoami shows your account. command -v oslab and oslab doctor check the helper.
- If the helper is missing, follow the local setup guide in the course repository.
- oslab start lab6 makes the lab folder. It keeps your work if you start again, and it does not change your current folder, so cd into it yourself.
- If you are resuming, look at the existing files before you run a command that writes to them.

Compare with the starting tree



private/record.txt is your private record.

shared/notice.txt is the notice that others should be able to read, once an administrator puts it in a searchable place.

The helper also writes a hidden file, .oslab-managed.json. Leave it as it is.

GUIDED

Permission basics

Task 3, guided part, with the instructor: fifteen minutes.

Step 1: who am I, and what do I own?

student@server: ~/oslab-work/lab6

```
$ id
$ ls -ld . private shared
$ stat -c '%A %a %U:%G %n' private/record.txt shared/notice.txt
```

Nothing is changed in this step. id shows your user and groups.

Each triple of the mode applies to the owner, the owning group and others. A directory's x bit is search, not running a program.

Step 2: octal and symbolic, on a demo file

student@server: ~/oslab-work/lab6

```
$ mkdir -p demo
$ printf 'demo content\n' > demo/note.txt
$ chmod 600 demo/note.txt
$ stat -c '%A %a %n' demo/note.txt
-rw------- 600 demo/note.txt
$ chmod u=rw,g=r,o= demo/note.txt
$ stat -c '%A %a %n' demo/note.txt
-rw-r----- 640 demo/note.txt
```

Use a separate demo file, so the lab files stay untouched.

The second mode, 640, lets the owning group read the file. It does not make any directory on the path searchable.

Step 3: look at the path

If namei is not installed, list each parent with ls -ld.

student@server: ~/oslab-work/lab6

```
$ namei -l "$PWD/demo/note.txt"
```

TRY

Observe

Does mode 644 on a file inside a 700 directory mean other users can reach that file?

PREDICT

Guess first, then try

Before you run the experiment. Ten minutes.

The prediction

PREDICTION

Write this down first

A directory has mode 600 and holds a mode-600 file that you own. Can you list the directory's names? Can you open the file through that directory? Explain the directory bits.

HOW

The rules

Alone, and without AI. On paper or in the course worksheet, before you run the command. You may compare with a neighbour for five minutes, but keep your own answer.

A wrong prediction costs nothing. It is marked for being made and for the evidence-based correction afterwards.

TASK 3

Private record and public-notice policy

Thirty-five minutes, from minute 35 to 70. Work alone.

Step 1 and 2: write the policy, then the modes

First, in words

- The owner reads and writes the private record. Nobody else reads it.
- Others may only read the notice, once it is in a searchable place.

Then, in modes

- Choose a mode for private/, record.txt, shared/ and notice.txt.
- Apply them only to these four paths.

save the result

```
$ stat -c '%A %a %U:%G %n' private private/record.txt \
$   shared shared/notice.txt | tee evidence/policy.txt
```

YOUR TURN

You choose the modes

Nobody gives you the numbers. Give the least access that still does the job.

Step 3: use the record as its owner

student@server: ~/oslab-work/lab6

```
$ cat private/record.txt
$ printf 'owner update\n' >> private/record.txt
$ if test -x private/record.txt; then printf 'unexpected execute bit\n'; else printf 'not executable\n'; fi
```

Read and append as the owner. Then check the file has no execute bit. A data file should not have one.

test -x asks: may I execute this? Do not run a data file to try its read permission.

Step 4: a directory you can read but not search

student@server: ~/oslab-work/lab6

```
$ mkdir -p demo/sealed
$ printf 'inside\n' > demo/sealed/item.txt
$ chmod 600 demo/sealed
$ ls demo/sealed | tee evidence/traversal.txt
$ cat demo/sealed/item.txt 2>&1 | tee -a evidence/traversal.txt
$ chmod 700 demo/sealed
$ cat demo/sealed/item.txt | tee -a evidence/traversal.txt
```

The failing cat is intentional. Record the failure and the repair in evidence/traversal.txt.

Restore search permission at once with chmod 700. Some ls variants also read file details and may print errors, so compare a plain list of names with real access.

Step 5: a peer's hypothetical access

Reason it, do not fake it

1. Look at every parent of the notice, including the private oslab-work folder.
2. A mode on the notice alone cannot turn a private workspace into a public place.
3. Do not widen your home folder or workspace to build a peer test.

HONESTY

Observed or reasoned?

A real test with another user needs an account the instructor prepared. Say clearly which statements you tested and which you only reasoned.

Complete when, and hints

DONE

Complete when

The modes match your policy. The traversal failure is explained and repaired. The peer reasoning covers the whole path.

Hints, one at a time

1. Keep file permissions apart from directory search.
2. Use stat and namei.
3. Restore search permission on the demo directory before you try the file again.

Three cases, three limits

Fifteen minutes, from minute 70 to 85.

What each case shows, and what it cannot show

Case	Evidence	Limit
Owner access	Read and append the private record, then inspect the modes	Does not simulate another user
Search failure	Before, failed and restored reads under demo/sealed	Shows traversal, not every ACL or capability effect
Peer access	A full-path explanation of the modes	Needs a prepared second account to confirm in practice

Default ACLs can change the mode of new files. Use getfacl if it exists and report any ACL you did not expect. Look at the final modes again after your tests.

If something does not work

Check who you are

Run whoami and id, and confirm you own the file before you change its mode.

chown is not for you

A normal user cannot give a file to another account. Do not try.

Stay within the core

Never edit sudoers or create accounts for this lab. See man chmod and man path_resolution.

INDIVIDUAL CHECKPOINT

Show what you can do alone

Fifteen minutes, from minute 85 to 100. No AI, no neighbours.

Why there is a checkpoint

It checks you, not the tools

Everything before it may use AI and help. The checkpoint shows what you understood without them.

A short question on paper

You answer one question on paper or on the course worksheet. Your earlier work does not need to be finished.

Practice question

The question on the next slide is public practice. A graded session may use a fresh private variant.

The question

CHECKPOINT

Practice question

A prepared shared directory has mode 711 and holds a file with mode 600, owned by another student. You know the file's name. Explain whether you can traverse the directory, list its names, and read the file.

What to give

1. The result or diagnosis.
2. The mechanism.
3. One observation that could check it.

The instructor collects your answer before discussing the public key.

Look back, then hand in

Ten minutes of correction, ten to clean up and submit.

Correct your prediction

1. Keep your original prediction in view. Under it, write confirmed or corrected.
2. Cite the test that shows it.
3. Explain the OS mechanism in three to five sentences.

ANSWER

Three questions

(a) Which directory bit caused the failed file read? (b) Why does a 644 notice not override a 700 parent? (c) Which observations were real tests, and which were reasoning?

Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab6/evidence"
$ cd "$OSLAB_WORKSPACE/lab6"
$ cp -- evidence/policy.txt evidence/traversal.txt "$SUBMISSION_REPO/lab6/evidence/"
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

Copy the two evidence files and two selected test records, not every terminal output. Check the final modes once more.

Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab6
├── evidence
│   ├── policy.txt
│   └── traversal.txt
└── README.md

1 directory, 3 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

How it is marked: 10 points

Evidence for the lab objectives	Points
Modes match the owned-file policy, with reasoning about the full path	3
Owner-access tests and the directory-search failure and repair	2
rwx and octal explained, and the limits of peer-access claims; original prediction and correction	2
Individual checkpoint: result or diagnosis, and mechanism	2
Concise, attributable evidence and the required artifacts	1

Other valid commands and names earn credit. A naming difference costs points only when it breaks the result. If you use AI, note one useful suggestion and how you verified it.

Common mistakes

Mistake	What you see	Fix
Mode 644 on a file in a 700 folder	Others still cannot reach it	Check every parent of the path
Directory with r but no x	Names listed, file not opened	Add x for search
Running a data file to test read	Permission denied, and a wrong lesson	Use cat, and test -x for execute
Widening your home to test a peer	A private folder opened to everyone	Reason it, or ask for a prepared account

BEYOND THE CORE

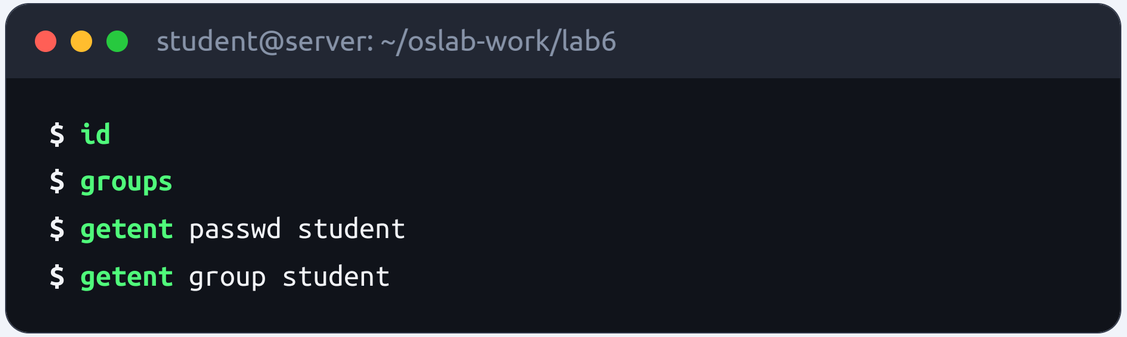
Accounts, ACLs and special bits

Optional extensions. They need prepared identities or a disposable virtual machine.

Lab 6

Accounts and groups

File	Holds
/etc/passwd	Accounts: name, uid, gid, home, shell
/etc/group	Groups and their members
/etc/shadow	Password hashes, readable by root only



Creating accounts and groups needs administrator rights. Do it only on a disposable virtual machine.

ACLs and special bits

ACL

ACLs

Extra rules for named users or groups, on top of the mode. getfacl shows them and setfacl changes them.

SPECIAL BITS

setuid and setgid

A program runs with its owner's or group's rights, not the caller's. On a directory, setgid makes new files inherit the group.

STICKY

Sticky bit

In a shared folder such as /tmp, only a file's owner may remove it, even if others can write the folder.

ls -l shows s and t in place of x, and chmod can set them with a fourth octal digit, for example 2755 or 1777.

Key takeaways

01

Three triples

Owner, group and others each get r, w and x. The first match decides.

02

Directory x is search

Without x on a folder you cannot reach what is in it, whatever the file's own mode says.

03

Check the whole path

The strictest folder between / and the file decides.

04

Observed or reasoned

Say which claims you tested and which you only worked out.

Next session

COMING UP

Lab 7: Bash scripting and server automation

Put commands in a script, pass it arguments, and make it safe to run.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 17, Protection.